

Machine Learning Project: S&P 500 Trading Signal Model

Type: Time-Series Classification (Multi-class: Buy/Sell/Hold)

Difficulty: Advanced (Conceptually challenging; requires strict adherence to validation protocols)

Primary Educational Goal: Learning robust time-series validation, preventing data leakage, and understanding the difference between theoretical accuracy and practical trading execution.

Objective

Build a machine learning classification model that analyzes historical market data to predict the optimal action (Buy, Sell, or Hold) for the S&P 500 ETF (SPY) over a defined future horizon (e.g., the next 5 trading days).

Crucial Note for Learners: The goal of this project is **not** to beat the market or build a highly profitable trading bot. The goal is to learn how to properly construct, evaluate, and backtest a time-series model on real-world, noisy data without cheating (data leakage).

Data Sources & Features

- **Primary Data:** Historical daily Open, High, Low, Close, and Volume (OHLCV) data for the SPDR S&P 500 ETF Trust (Ticker: SPY). This can be easily retrieved using the yfinance Python library.
- **Feature Engineering (Required):**
 - **Returns:** Raw prices are non-stationary and must be converted to percentage returns (e.g., daily returns, 5-day rolling returns).
 - **Technical Indicators:** Incorporate at least three technical indicators (e.g., Moving Averages, RSI, MACD, Bollinger Bands) derived from the price and volume data.
 - **Volatility Measures:** Include a measure of historical volatility (e.g., rolling standard deviation of returns).
 - **Constraint:** All features must be strictly lagged. A prediction for tomorrow's action can only use data available up to today's close.

Target Definition (The "Buy, Sell, Hold" Logic)

You must convert continuous future returns into discrete classes. Instead of predicting the exact price, define thresholds based on the expected percentage return over a future period (e.g., the next 5 days).

Example Thresholds (Learners can adjust these):

- **Buy (Class 1):** Future 5-day return is $> +1.0\%$
- **Sell (Class -1):** Future 5-day return is $< -1.0\%$
- **Hold (Class 0):** Future 5-day return is between -1.0% and $+1.0\%$

Key Requirements & Guardrails

1. **Strict Chronological Splitting:** You absolutely cannot use random train/test splits (e.g., standard `train_test_split`). You must split the data chronologically (e.g., Train: 2010-2020, Validation: 2021-2022, Test: 2023). Alternatively, use `TimeSeriesSplit` for cross-validation.
2. **Addressing Class Imbalance:** The stock market naturally trends upwards over long periods. Your classes will likely be imbalanced (more "Buys" or "Holds" than "Sells"). You must implement a strategy to handle this (e.g., SMOTE, class weighting, or adjusting your threshold boundaries).
3. **Model Selection:** Start with interpretable models suitable for tabular classification (e.g., Logistic Regression, Random Forest, or XGBoost/LightGBM). Do not start with complex Deep Learning architectures.
4. **No Look-Ahead Bias:** Your code will be audited for look-ahead bias (using future information to predict the past).

Evaluation & Deliverables

Your success will be measured by your methodology, code quality, and interpretation of results, not by how much "money" your model makes.

Required Deliverables:

1. **Exploratory Data Analysis (EDA) Notebook:** Visualizations of feature distributions, correlations between indicators, and an analysis of target class distribution.
2. **The Modeling Notebook (The Core):** A clean, documented Jupyter Notebook showing the data pipeline, feature creation, model training, and hyperparameter tuning.
3. **Realistic Backtest:**
 - You must use a dedicated backtesting library (such as `vectorbt`, `backtrader`, or `zipline`) to simulate trading based on your model's signals.
 - **Mandatory:** Your backtest must include simulated transaction fees/slippage (e.g., 0.1% per trade). A model that trades every day will often lose to fees; this is a critical lesson.
4. **Benchmark Comparison:** Your final output must plot your model's simulated portfolio value against a simple "Buy and Hold" strategy of SPY.
5. **"Lessons Learned" Report:** A brief written section analyzing where the model failed, why it underperformed (or outperformed) the baseline, and a discussion on the difficulty of predicting financial markets.

Realistic Expectations

- **Accuracy:** A model achieving ~35-40% accuracy across three balanced classes is performing normally for this type of noisy data. If you see 80% accuracy, you almost certainly have a data leak.
- **Profitability:** It is highly likely that your model, once transaction fees are applied, will underperform the "Buy and Hold" benchmark. This is expected and perfectly acceptable for this project. The grade is based on proving *why* it underperformed and demonstrating correct machine learning practices.